A Crash-Proof Team of Coding Agents: How It's Built

The engineering companion: the settings, the tuning, the governance plumbing, and the operational details the flagship's story rides on.

This is the mechanism half. The story, the claims, and the fine print (one engineer, a cheap fast model, single-digit runs, point estimates) live in the flagship. The cost measurements live in the economics companion, Mechanics Cost Cents, Behavior Costs Dollars. All of it applies verbatim here.

The flagship tells you a worker died mid-task and the session walked away from the crash. This article is where that stops being a trick: the settings that make it true, the retry numbers, the queue plumbing, and one whole design we built, proved live, and then retired because a measurement said we should. Most of what follows was shaped by a run that went wrong in a specific way. The runs are named as we go, because the scars are the documentation.

Vocabulary, in one breath, for anyone arriving cold: this system runs headless Claude Code sessions under Temporal, a durable-execution engine. A **workflow** is the deterministic, replayable layer that decides what happens next. An **activity** is a sealed step that may do real-world work. A **chunk** is one bounded activity-run of the agent, capped at a fixed number of turns. A **heartbeat** is the liveness pulse a running activity sends. The flagship earns each of those words. This article spends them.

The Rivets on a Chunk

The flagship opens the box: one chunk is a single ordinary function, and what comes back is a typed record the workflow can only read fields on: the wall that keeps the agent's chaos out of the replayable layer. What the flagship does not show are the rivets.

The launch is a short list of settings, driven through the Python `claude-agent-sdk` (`ClaudeAgentOptions`, built in `src/activities.py`):

- **Configuration from the project directory only** (`setting_sources=["project"]`), never the machine's own user config. This is the quiet heavy lifter, and it is why the policy is true rather than hopeful: the rules ride inside the workspace, so the same agent runs identically on every worker on every machine.
- **Resume the prior session** (`resume=<the chained session ID>`).
- **Cap the turns** (`max_turns`; this repo ships 40), so a chunk stops cleanly with its transcript intact.
- **Auto-accept file edits** (`permission_mode="acceptEdits"`), and gate every other tool behind an explicit allow-list: Read, Write, Edit, Glob, Grep, Bash, TodoWrite, Skill.
- **Force the closing report to match a schema** so the workflow reads a real pass/fail value instead of parsing prose. The schema is three fields (`summary`, `files_created`, `tests_passed`), and the last one is the merge switch.

The policy itself is a small settings file (`settings.json`). Humans control it. The rules live in version-controlled harness code, and the workspace re-stamps them fresh at the start of every chunk:

```
{
  "permissions": {
    "deny": ["Bash(rm -rf:*)", "Bash(sudo:*)", "Bash(git push:*)"]
  },
  "hooks": {
    "PostToolUse": [
      { "matcher": "*", "hooks": [{ "type": "command", "command": "cat >> .claude/hook-
log.jsonl" }] },
      { "matcher": "*", "hooks": [{ "type": "command", "command": "python3 .claude/flag-
rules.py" }] }
    ]
  }
}
```

The deny list is the org floor the flagship reads as architecture; the two hooks are the audit pair. The first appends every tool call to `hook-log.jsonl` as it happens; the second runs the team's watched-rule flags. (`flag-rules.py` is a small stdin-JSON matcher driven by the lane's committed `rules.json`; both live under `teams/<team>/.claude/` in the repo. Without both files the hook exits silently and flags nothing.) Beyond the settings file, each chunk also installs the team's **skills** (real playbook files the agent discovers and invokes through Claude Code's own Skill tool) and a project memory, a `CLAUDE.md`, that points the agent at them. It is a stock Claude Code project, assembled fresh every chunk; Temporal does not replace that ecosystem: it decides when the project runs, where, and which skills come with it.

How a chunk ends. Three typed exits: resume, continue, or stop loudly. Nothing ends silently. Color key, used across this family: indigo = the durable machinery · amber = judgment (agent or human) · purple = the durable record · green = a good exit · red = failure · dashed = crosses a team or a poll boundary.

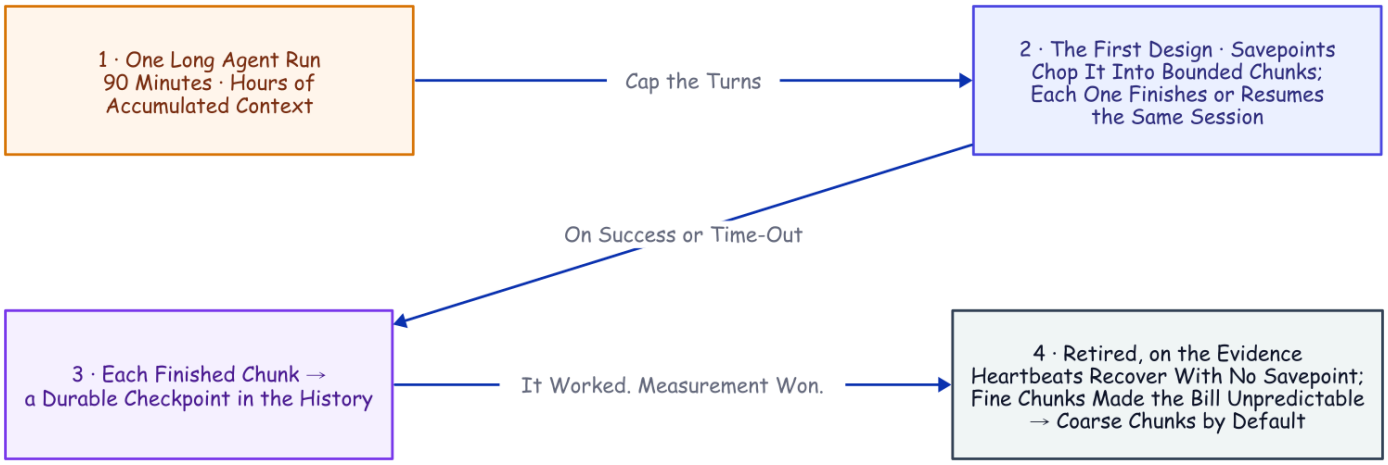
When a chunk fails, the function raises a typed error and lets the workflow's declared retry policy handle the backoff. The tuning is deliberate: the agent chunk's policy (5s, doubling to a 2-minute cap, 6 attempts) is shaped for the API failure a headless run actually meets rather than the rare crash, while the plain GitHub steps around it get a plainer policy: a refused merge is not an overload. That first 5s backoff quietly covers two deaths with one recovery: when the agent process dies but the worker lives (its subprocess SIGKILLed, or an API error ending the run), the activity's retryable error resumes the same session from the heartbeat in seconds; when the whole worker dies, a fresh one rebuilds from history and resumes from the same heartbeat, caught by the two-minute liveness timeout. Only detection speed and worker survival differ, and both were killed on purpose to prove it (`deploy/child-kill-recovery.sh`).[1]

The Detour We Measured Our Way Out Of

There is an older answer buried in this codebase, and it is worth showing precisely because it *worked*, until measurement talked us out of it. One reminder first, because this section turns on it: every heartbeat carries the live session ID, so when a worker dies mid-chunk the retry reads the last pulse and resumes the same session instead of starting over. That is heartbeat recovery.

An activity's result is all-or-nothing: nothing lands in the history until the attempt finishes. Wrap a ninety-minute agent run in one activity and a crash at minute eighty-nine erases everything the workflow could see. So the first design chopped the run into **savepoints** on that same turn cap. A capped run either finishes or

runs out of turns mid-task, and in that second case the transcript survives, resumable by ID. Chain those together and you get a chunked session. Each activity runs one bounded piece, and every completed piece writes progress, cost, and the session ID into the history. It worked: we killed a worker in the middle of the second of four chunks on a real to-do-app build. It recovered on a restarted worker and finished all ten tests, one session end to end. (That recovery was a savepoint-era run, recorded in the lab notebook this project was curated from. The flagship's headline kill is a later, harder one: the worker died before *any* chunk completed and the session still resumed; that run's receipt is in the evidence repository.)



The Savepoint Detour. The design was sound and the recovery was real. Two later facts retired it: heartbeat recovery survives a crash with no completed chunk at all, and cutting the run into many small pieces makes the bill unpredictable.

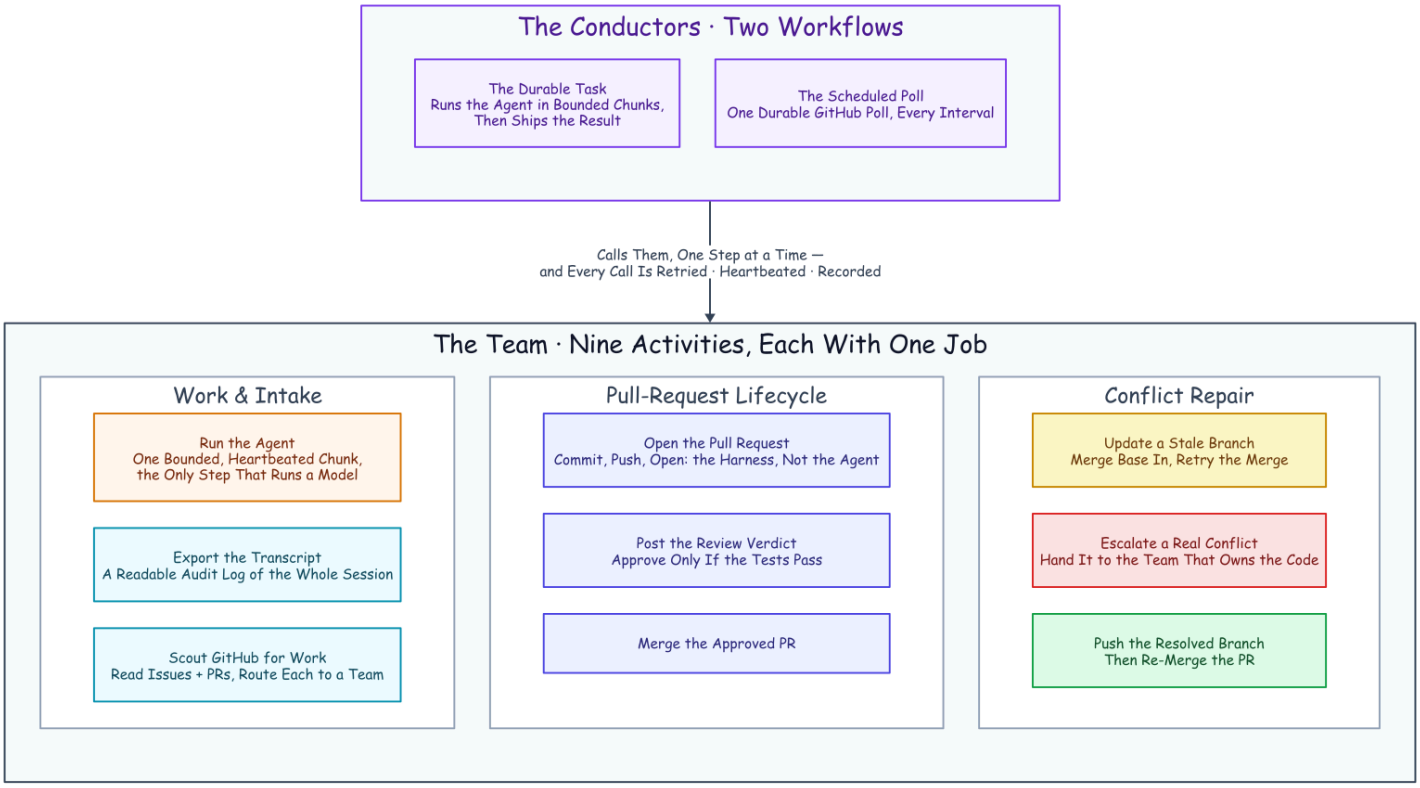
But every completed-chunk boundary is another resume: an agent invocation that re-establishes the growing conversation before it does any new work. Once heartbeat recovery existed, those boundaries were buying a durability we already had. And they were quietly doing something worse to the cost. Fine-chunking the same task later ran $0.034 to $2.13 on identical code, a spread driven by agent behavior rather than any cache tax. That measurement, and the experiment that fooled us before it, live in the economics companion, Mechanics Cost Cents, Behavior Costs Dollars.

Three Verbs, a Few Dozen Lines

The flagship shows the three verbs the surviving boundaries buy (query, steer, cancel) and prices the hand-build alternative in a table. The mechanics behind those verbs are smaller than they sound, a few dozen lines of workflow and activity code. **The status query answers from the workflow's own replicated state**, so it needs no database and works even while the agent is mid-tool-call. **A steer is a signal the workflow folds into the *next* chunk's prompt**, and one that lands during the final chunk buys an extra chunk rather than vanishing. **A cancel rides the heartbeat channel**: the activity asks the running agent to stop, the agent exits cleanly rather than burning tokens as an orphan, and the job records a canceled state you can query. (The query and steer demos are recorded in the lab notebook this project distills.)

Why Adding a Teammate Is Boring

Two conductor workflows, nine single-purpose activities in the measured runs, each a sealed box handing back typed data: that is the org chart the flagship names.



The Team of Activities. Two workflows conduct nine one-job specialists (nine at the time of the measured runs; the repo now ships twelve). Read the columns as sub-teams: doing the work and finding it; the pull-request lifecycle; and repairing a conflict. Every box is a durable activity, so every box is retried, heartbeated, and recorded without anyone writing that code.

Adding a member is mechanical, and that is what makes this a design rather than a script: one new function that does the real-world thing and returns typed data, a declared retry policy, and a named moment in the workflow to run it. Everything else (crash recovery, backoff, cancellation, the audit trail) arrives from the harness, not from anything you wrote. The hard, distributed-systems parts are already solved; the only thing you add is the one honest step.

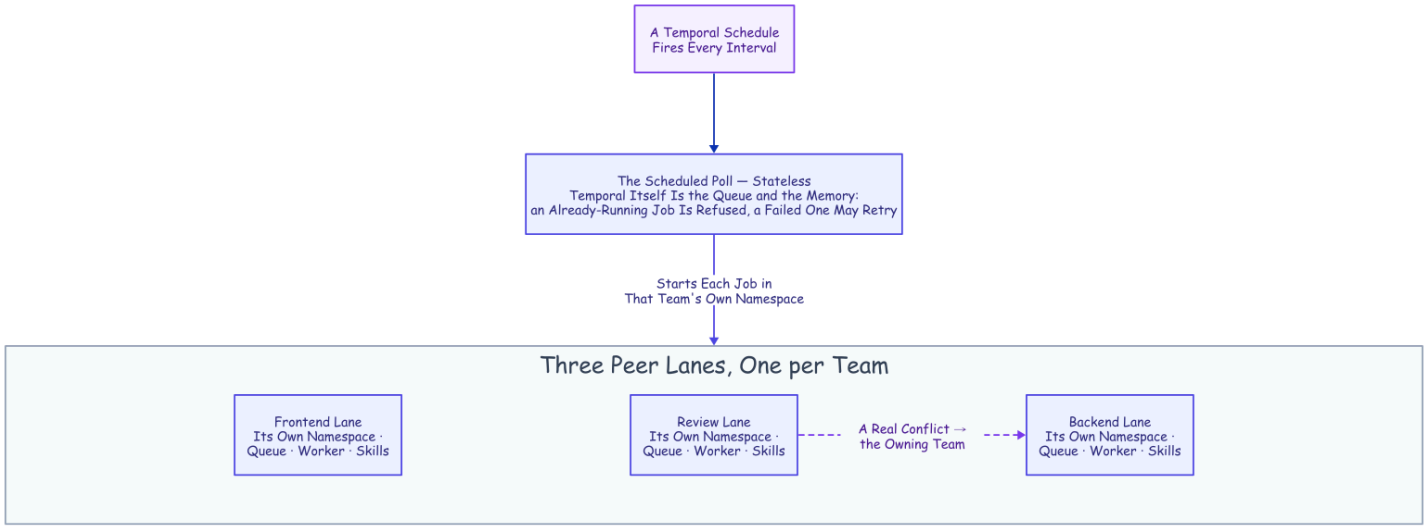
Lane Plumbing

The flagship establishes what a lane is (a namespace, a queue, a worker, a skill bundle) and why the queue, not the prompt, is the identity. Here is the plumbing underneath that claim.

The topology lives in version control. The lanes are created from a script at startup (`deploy/stack-up.sh`, which runs `temporal operator namespace create` per team folder; namespaces must exist before a lane's worker starts), discovered from the `teams/` folders (six today; backend, frontend, and review in the

measured runs), never clicked into existence. Each team's folder carries its mandate and its own copies of the disciplines it runs (test-first, review-your-own-diff, the required report format) tuned to the lane rather than cloned across it. The workspace binds to that folder *live*: the mandate is imported and the skills are linked, so the owning team's edits reach the very next chunk, while only the policy files are stamped per chunk as an immutable snapshot.

Governance layers, precisely. An identical org-wide deny floor governs every lane. On top of that floor, each team commits its own watched rules, its own finish gate, and any extra denies it chooses. The review lane also denies `git commit`: reviewers read, they do not write. And the load-bearing rule in the bundles is cross-layer: a backend change is obliged to run the frontend and end-to-end suites too, and the review lane will not approve until that evidence comes back green. A backend agent and a reviewer differ by discipline and by what their own team chooses to enforce, never by whether they are governed at all.



Trusted by the Queue, Not the Prompt. Three identical peer lanes; only the queue they poll differs. The scheduled poll and the cross-lane escalation both reach another namespace the same small way: a client call made from inside an activity.

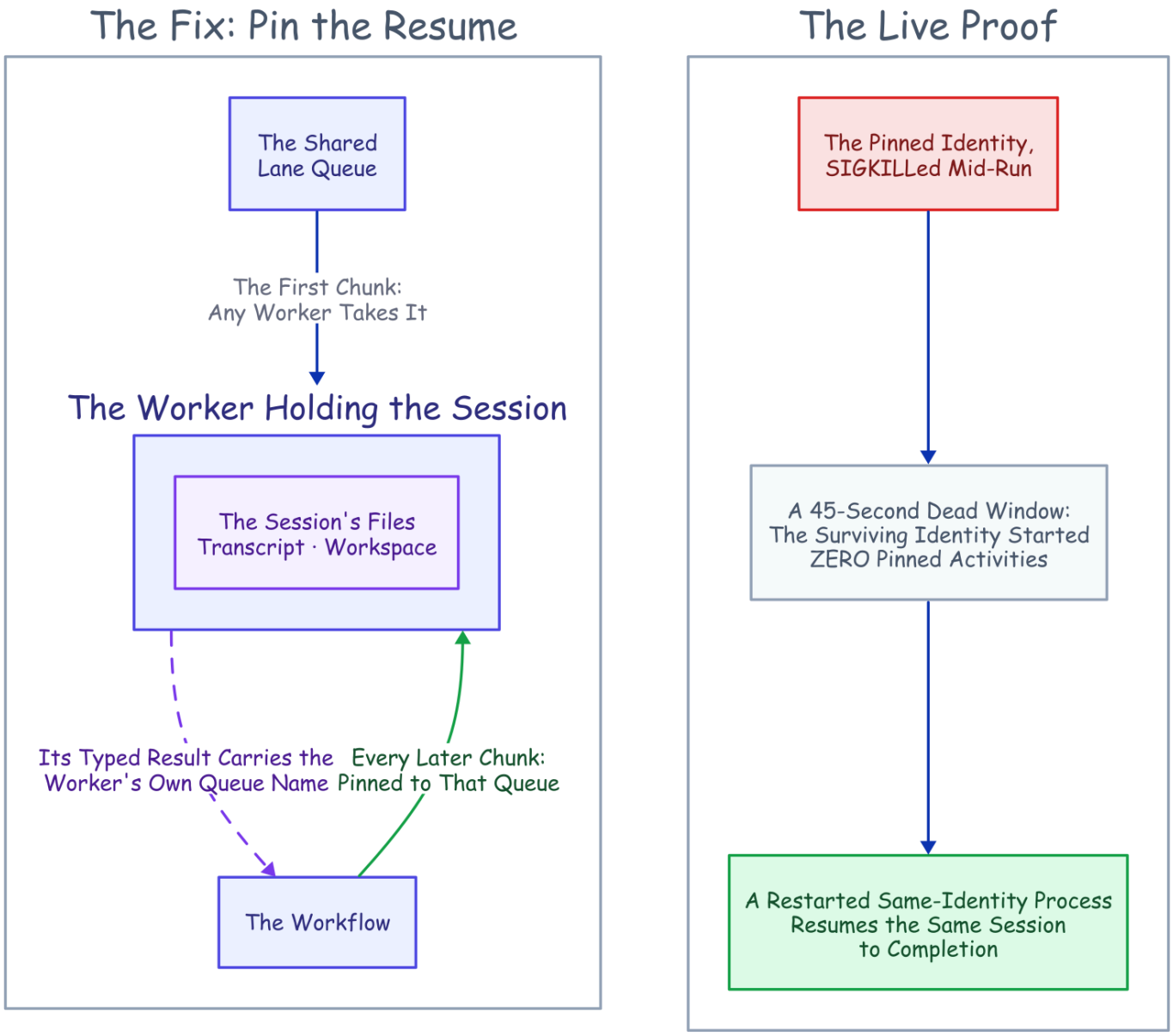
Backpressure we met live on the thirteen-issue fleet run of 2026-08-06: eight reviews queued behind hour-class agent chunks, a busy lane head-of-line blocking its own fast activities (post, merge, push). The answer is the queue itself plus a cap, tunable, on how many chunks a single worker will run at once. You scale a hot lane by adding workers to it.

The One Place Durability Isn't Symmetric

Adding workers surfaces the one place the design has to be careful, because the durability is not fully symmetric. Temporal can recover the *workflow* on any worker, but a chunk's session lives in *local files*: the transcript under the Claude projects directory keyed by the working directory, and the workspace under the temp directory keyed by the workflow ID. On a single-worker lane that is invisible. Put two workers on a lane and a resumed chunk could land on the worker that never held the session, and the resume finds nothing.

The fix is a **sticky per-worker queue**, opt-in so a single-worker lane is unchanged. Each worker listens on two queues: the shared lane queue, and its own stable queue named for the host (stable so it survives a process restart, which is exactly when the local files are still on disk). The first chunk runs on the lane queue, any worker takes it, and it reports its per-worker queue back as typed data. The workflow reads that off the chunk result and pins every later chunk and the transcript export to it, deterministically, because the queue name came across the seam like everything else. A restart on the same host now resumes on the right box, and a chunk can no longer silently land where the session is absent. (In the repo: set `WORKER_AFFINITY=1`; the per-worker queue is named `<lane>-w-<hostname>` in `src/shared.py`.)

The routing has been proven twice. Offline, on the time-skipping test server. And live: two worker identities on a real server; the pinned identity SIGKILLed mid-run; a 45-second dead window in which the surviving identity started exactly zero pinned activities; and a restarted same-identity process resuming the same session to completion. Every check was read from the event history.

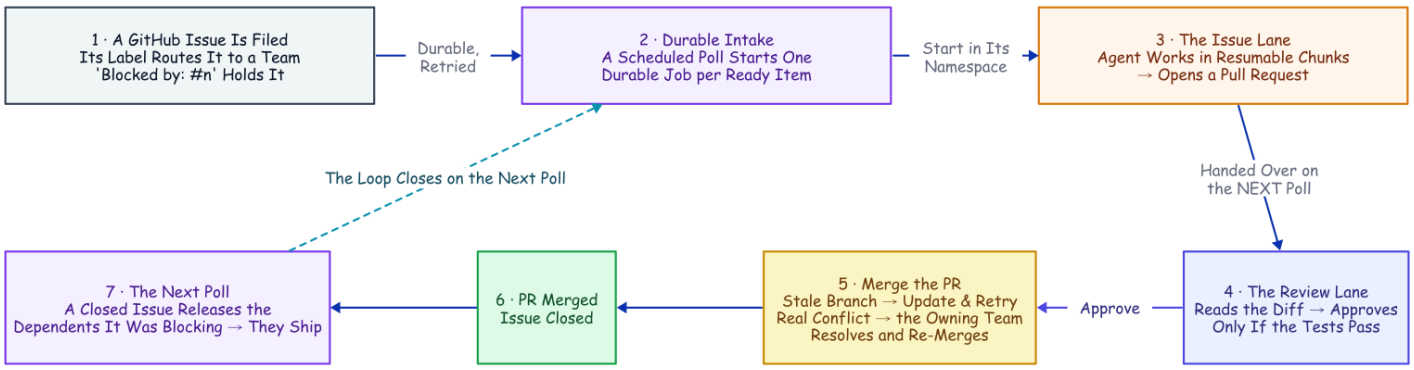


The One Place Durability Isn't Symmetric. The queue name crosses the seam like everything else, so the resume lands where the session lives. Two identities, one filesystem; the machine-loss run is the separate receipt.

The honest edge was that the live run's two identities shared one filesystem, so true machine-loss (the local files gone with the machine) still wants shared storage for the transcript and workspace under the same key. That debt has since been paid: `deploy/machine-loss-live.sh` stages A's workspace and transcript to a shared-storage directory, deletes machine A's roots outright, and machine B resumes the same session from the heartbeat ID, seven checks green (`deploy/machine-loss-results.md`, 2026-08-20). The harness still does not ship the sync itself; the staging copy stands in for the shared mount a production deployment would run.

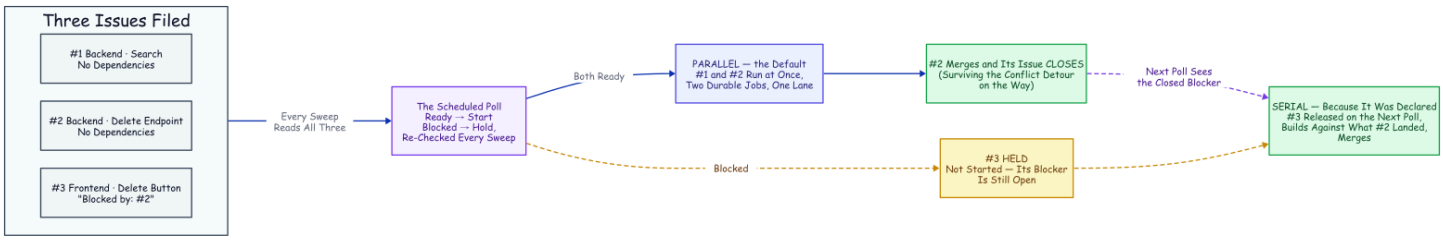
Pipeline Details the Story Skips

The flagship walks the filed-issue-to-merged-PR loop. The details worth having when you build it:



From Filed Issue to Merged PR. Every box past the filed issue is a real activity from the team. The conflict detour and the unblock loop are not special cases; they are the same durable machinery running one more time.

Routing goes by the issue's `team/…` label, with a catch-all lane when it carries none, so an unlabeled issue is never silently dropped. **Sequencing** honors *blocked by* in either dialect: a line in the issue's body or a `blocked-by` label. **Deduplication** carries a scar. The same 2026-08-06 fleet run taught the lesson the hard way: under the stricter refuse-everything reuse policy, one transient error deadlocked an issue forever, because the failed job's ID could never be started again. The fix is the policy the repo ships, and it is the server's, not the poller's: each job's ID derives deterministically from the issue, started with an "allow duplicates only after failure" policy (Temporal's `WorkflowIDReusePolicy.ALLOW_DUPLICATE_FAILED_ONLY`), so a running or completed job refuses a second start while a failed one may retry on a later sweep: the reason the poller keeps no memory of its own. **Namespace crossings** (the scheduled intake and the conflict escalation both) happen from *inside an activity*, which is allowed to do arbitrary I/O: open a client to the target namespace and start the job there, idempotent across the boundary because of the deterministic ID.



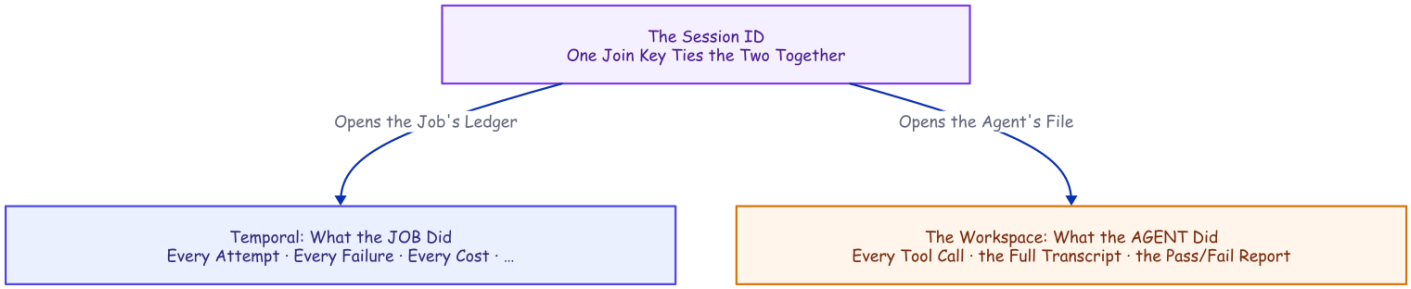
The Two Speeds. Parallel is the default; serial is declared. A blocked issue is held on every sweep and released only when its blocker closes, not when it merely looks done.

The commit history is the agent's own. Each working lane's mandate says commit at meaningful checkpoints (a phase completed, a suite gone green) with messages that say why, and the PR activity pushes that full history rather than squashing it. The harness's final commit only sweeps whatever the agent left uncommitted, and "nothing to ship" is judged against the base branch, never against a clean status.

And the red verdict does not have to be a dead end. With the optional fix loop on, a failed review (or a human's Request Changes) hands the PR back to the owning lane to fix, re-validate, and re-review, re-asking the human when the merge gate is also on, bounded to a few rounds; the mechanics and the live run are in the companion piece.

Every Run Leaves Evidence

The flagship's denouement shows the two planes of evidence joined on the session ID. The operational half of that claim is hygiene, and two details are worth stealing.

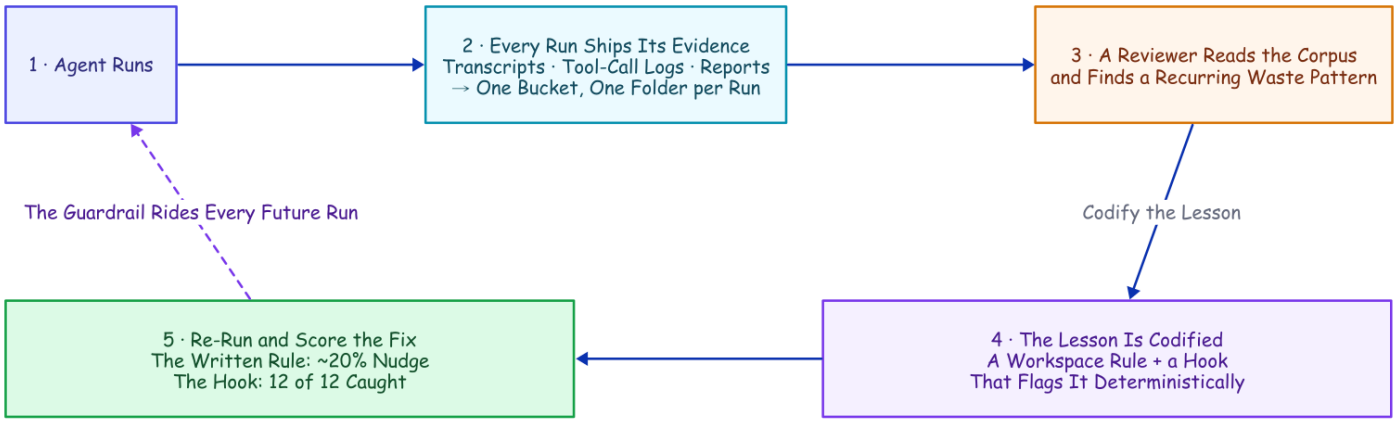


Every Run Leaves Evidence. Ask what the job did and you read Temporal's history; ask what the agent did and you read the workspace's package. One session ID opens both records.

The activity clones the repository with a token and then scrubs that token out of the remote, so the agent cannot recover the clone credential from the repo's own config. And the token is withheld from the agent subprocess's environment entirely, so the credential the push activities use never exists anywhere the agent can look. **Three layers, one claim**: the deny rule blocks the push command, the scrub keeps the token out of the checkout, and the environment filter keeps it out of the process. One layer this design still owes: the hook log lives inside the workspace the agent can edit, so a determined agent could tamper with its own trail mid-chunk; shipping the log outside the workspace is the hardening that closes it. And before any push, the harness excludes its own scratch (the configuration folder, the memory file, the report, every audit log) from the commit, so the control plane's policy and its paper trail never ride inside the product it ships. The agent writes the code; the control plane is the only thing that touches the outside world.

The Rule Nudged; the Hook Was Law

The flagship tells the learn-loop story: the `ls`-recheck waste, the rule versus the hook. Here are the numbers behind it, because the aggregate teaches more than the slogan. (Shipping the corpus is deliberately dumb: an object-store copy of each run's export folder, keyed by session ID, not part of the harness itself; a frozen example is `deploy/fleet-corpus-2026-08-06.tar.gz`.)[2]



The Corpus Improves the System. Runs become a corpus in a bucket; a reviewer finds a recurring waste; the lesson becomes a guardrail that rides every future run; a re-run scores it.

Across the six corpus runs, the leanest finished in eight or nine tool calls; the worst burned twenty-four, dominated by re-checking work already done. After codifying the lesson both ways, the re-run scored it: the written rule produced a modest ~20% drop in the targeted pattern (2.5 to 2.0 instances per run, a half-instance change inside run-to-run noise) and one run ignored it entirely, while the total tool count did not fall at all (it rose, 15.5 to 18.5, swamped by ordinary nondeterminism). The hook flagged every remaining instance, twelve out of twelve, regardless of whether the agent complied. The prompt rule is probabilistic; the hook is deterministic. The useful result is not that the agent improved on average: it is that the mistake is now caught on every run, and the guardrail rides every future retry because it lives in the workspace bootstrap, not in anyone's memory.

The Family

- A Crash-Proof Team of Coding Agents, the flagship: the kill, the team, the conflict that resolved itself
- Mechanics Cost Cents, Behavior Costs Dollars: every boundary priced; the family's cost ledger
- Flag, Block, or Beg: the tool-call boundary, measured
- Done Is Not a Claim: the finish boundary, measured
- The Agent Grades Its Own Homework: the merge switch's boolean vs ground truth
- The Human Is a Durable Object: the human merge gate on four workflow primitives

Prerequisites, if you want to run the system yourself: a Temporal dev server (`temporal server start-dev`), `uv`, a logged-in `claude` CLI, and a `GITHUB_TOKEN` with contents and pull-request write scopes. `deploy/quickstart.sh` checks the first three; the token matters only once the GitHub pipeline runs.

The code and every results file cited live in the repository, github.com/thumarrushik/crash-proof-team-of-coding-agents: the system under `src/`, the lanes under `teams/`, the runs and `quickstart.sh` under `deploy/`.

Personal project; views are my own and not my employer's. Not affiliated with or endorsed by Anthropic or Temporal.

---

1. The workspace policy is human-committed in each team's folder (the deny rules, the two `PostToolUse` hooks, a `Stop`-hook phase gate added after these runs, and the lane's watched rules) and the harness stamps it into the workspace each chunk, injecting absolute-path denies that protect the team folders themselves. The project-only config load, the accept-edits mode with an explicit tool allow-list, and the schema-required report live in the chunk activity. The per-call retry policy (5s, doubling to a 2-minute cap, 6 attempts), the two-minute heartbeat timeout, and the fifteen-minute chunk ceiling are set where the workflow invokes the chunk. ↩

2. The learn-loop, 2026-07-15: the six cost-experiment runs' transcripts, tool logs, and reports were pushed to the corpus bucket; a reviewer identified redundant directory-listing re-orientation; the lesson was codified as a workspace instruction rule plus a `PostToolUse` flag hook; a re-run scored the modest rule effect and the deterministic hook (12 flags). Full results in the evidence repository. ↩